

Exercise 07

Index No: 25000799

Registration No: 2025/CS/079

(01)

(a)

stack.h

```
#ifndef STACK_H
```

```
#define STACK_H
```

```
#define MAX_SIZE 1000
```

```
typedef struct {  
    int data[MAX_SIZE];  
    int top;  
} IntStack;
```

```
void init_stack(IntStack *stk);  
int is_empty(IntStack *stk);  
int is_full(IntStack *stk);  
void push(IntStack *stk, int val);  
int pop(IntStack *stk);
```

```
#endif
```

stack.c

```
#include <stdio.h>
```

```
#include "stack.h"
```

```
void init_stack(IntStack *stk) {  
    stk->top = -1;  
}
```

```
int is_empty(IntStack *stk) {  
    return stk->top == -1;  
}
```

```
int is_full(IntStack *stk) {  
    return stk->top == MAX_SIZE - 1;  
}
```

```
void push(IntStack *stk, int val) {  
    if (is_full(stk)) {  
        printf("Stack Overflow\n");  
        return;  
    }  
    stk->data[++(stk->top)] = val;  
}
```

```
int pop(IntStack *stk) {  
    if (is_empty(stk)) {  
        printf("Stack Underflow\n");  
        return -1;  
    }  
    return stk->data[(stk->top)--];  
}
```

```
ChamalInduwara@MacBook-Air : ~/Downloads  
$ gcc q1_part_a.c stack.c
```

```
ChamalInduwara@MacBook-Air : ~/Downloads  
$ ./a.out  
Pushing 10 to stack...  
Pushing 20 to stack...  
Popped: 20  
Popped: 10
```

```

main.c
#include <stdio.h>
#include "stack.h"

int main() {
    IntStack num_stack;
    init_stack(&num_stack);

    printf("Pushing 10 to stack...\n");
    push(&num_stack, 10);
    printf("Pushing 20 to stack...\n");
    push(&num_stack, 20);

    printf("Popped: %d\n", pop(&num_stack));
    printf("Popped: %d\n", pop(&num_stack));

    return 0;
}

```

(b)

```

#include <stdio.h>
#include <string.h>
#include "stack.h"

int main() {
    IntStack operand1, operand2, result;
    init_stack(&operand1);
    init_stack(&operand2);
    init_stack(&result);

    char num1[MAX_SIZE], num2[MAX_SIZE];

    printf("Enter first operand: ");
    if (scanf("%s", num1) != 1) return 1;

    printf("Enter second operand: ");
    if (scanf("%s", num2) != 1) return 1;

    for (int i = 0; i < strlen(num1); i++) {
        push(&operand1, num1[i] - '0');
    }

    for (int i = 0; i < strlen(num2); i++) {
        push(&operand2, num2[i] - '0');
    }

    int carry = 0;

    while (!is_empty(&operand1) || !is_empty(&operand2) || carry) {
        int d1 = is_empty(&operand1) ? 0 : pop(&operand1);
        int d2 = is_empty(&operand2) ? 0 : pop(&operand2);

        int sum = d1 + d2 + carry;
        push(&result, sum % 10);
        carry = sum / 10;
    }
}

```

```

ChamalInduwara@MacBook-Air : ~/Downloads
$ gcc q1_part_b.c stack.c

ChamalInduwara@MacBook-Air : ~/Downloads
$ ./a.out
Enter first operand: 12
Enter second operand: 14
Result: 26

```

```

    printf("Result: ");
    while (!is_empty(&result)) {
        printf("%d", pop(&result));
    }
    printf("\n");

    return 0;
}

```

(c)

```

#include <stdio.h>
#include <stdlib.h>

```

```

#define CAPACITY 100

```

```

typedef struct {
    char elements[CAPACITY];
    int cursor;
} CharStack;

```

```

void initialize(CharStack *stk) {
    stk->cursor = -1;
}

```

```

int check_full(CharStack *stk) {
    return stk->cursor == CAPACITY - 1;
}

```

```

int check_empty(CharStack *stk) {
    return stk->cursor == -1;
}

```

```

void push_char(CharStack *stk, char element) {
    if (check_full(stk)) {
        printf("Stack Overflow\n");
        return;
    }
    stk->elements[++(stk->cursor)] = element;
}

```

```

char pop_char(CharStack *stk) {
    if (check_empty(stk)) {
        printf("Stack Underflow\n");
        return '\0';
    }
    return stk->elements[(stk->cursor)--];
}

```

```

int main() {
    CharStack stk;
    initialize(&stk);

    printf("Pushing 'A' to stack.\n");
    push_char(&stk, 'A');
    printf("Pushing 'B' to stack.\n");
    push_char(&stk, 'B');

    printf("Popped: %c\n", pop_char(&stk));
    printf("Popped: %c\n", pop_char(&stk));
}

```

```

ChamalInduwara@MacBook-Air : ~/Downloads
$ gcc q1_part_c.c

```

```

ChamalInduwara@MacBook-Air : ~/Downloads
$ ./a.out
Pushing 'A' to stack.
Pushing 'B' to stack.
Popped: B
Popped: A

```

```

    return 0;
}

```

(d)

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

```

#define MAX_LEN 1000

```

```

typedef struct {
    char items[MAX_LEN];
    int top_idx;
} CharStack;

```

```

void init_stack(CharStack *stk) {
    stk->top_idx = -1;
}

```

```

int is_empty(CharStack *stk) {
    return stk->top_idx == -1;
}

```

```

int is_full(CharStack *stk) {
    return stk->top_idx == MAX_LEN - 1;
}

```

```

void push(CharStack *stk, char ch) {
    if (is_full(stk)) return;
    stk->items[++(stk->top_idx)] = ch;
}

```

```

char pop(CharStack *stk) {
    if (is_empty(stk)) return '\0';
    return stk->items[(stk->top_idx)--];
}

```

```

int is_matching_pair(char open_symbol, char close_symbol) {
    if (open_symbol == '(' && close_symbol == ')') return 1;
    if (open_symbol == '{' && close_symbol == '}') return 1;
    if (open_symbol == '[' && close_symbol == ']') return 1;
    return 0;
}

```

```

int main() {
    CharStack bracket_stack;
    init_stack(&bracket_stack);

    char input[MAX_LEN];
    printf("Enter string: ");
    if (scanf("%s", input) != 1) return 1;

    for (int i = 0; i < strlen(input); i++) {
        char current = input[i];

        if (current == '(' || current == '{' || current == '[') {
            push(&bracket_stack, current);
        }
    }
}

```

```

ChamalInduwara@MacBook-Air : ~/Downloads
$ gcc q1_part_d.c

```

```

ChamalInduwara@MacBook-Air : ~/Downloads
$ ./a.out
Enter string: {}
All brackets are in order

```

```

ChamalInduwara@MacBook-Air : ~/Downloads
$ ./a.out
Enter string: [{]}
Error: symbols do not match

```

```

    else if (current == ')' || current == '}' || current == ']') {
        if (is_empty(&bracket_stack)) {
            printf("Error: symbols do not match\n");
            return 0;
        }
        char top_char = pop(&bracket_stack);
        if (!is_matching_pair(top_char, current)) {
            printf("Error: symbols do not match\n");
            return 0;
        }
    }
}

if (is_empty(&bracket_stack)) {
    printf("All brackets are in order\n");
} else {
    printf("Error: symbols do not match\n");
}

return 0;
}

```

(02)

(1)

```
#include <stdio.h>
```

```
#include "q2_queue_single_b.h"
```

```

int main() {
    PriorityQueue pq;
    init_pq(&pq);

    printf("Enqueue Task 2 with Medium Priority (2)\n");
    pq_enqueue(&pq, 2, 2);
    printf("Enqueue Task 1 with High Priority (3)\n");
    pq_enqueue(&pq, 1, 3);
    printf("Enqueue Task 3 with Low Priority (1)\n");
    pq_enqueue(&pq, 3, 1);
    printf("Enqueue Task 4 with High Priority (3)\n");
    pq_enqueue(&pq, 4, 3);

    while (!is_pq_empty(&pq)) {
        printf("Processing Task: %d\n", pq_dequeue(&pq));
    }

    return 0;
}

```

```
#include <stdio.h>
```

```
#include "q2_queue_single_b.h"
```

```

void init_pq(PriorityQueue *pq) {
    pq->count = 0;
}

```

```

int is_pq_empty(PriorityQueue *pq) {
    return pq->count == 0;
}

```

```

ChamalInduwara@MacBook-Air : ~/Downloads
$ gcc q2_main_single_b.c q2_queue_single_b.c

```

```

ChamalInduwara@MacBook-Air : ~/Downloads
$ ./a.out

```

```

Enqueue Task 2 with Medium Priority (2)
Enqueue Task 1 with High Priority (3)
Enqueue Task 3 with Low Priority (1)
Enqueue Task 4 with High Priority (3)
Processing Task: 1
Processing Task: 4
Processing Task: 2
Processing Task: 3

```

```

int is_pq_full(PriorityQueue *pq) {
    return pq->count == MAX_CAPACITY;
}

void pq_enqueue(PriorityQueue *pq, int val, int priority) {
    if (is_pq_full(pq)) return;
    pq->tasks[pq->count].val = val;
    pq->tasks[pq->count].priority = priority;
    pq->count++;
}

int pq_dequeue(PriorityQueue *pq) {
    if (is_pq_empty(pq)) return -1;

    int highest_idx = 0;
    for (int i = 1; i < pq->count; i++) {
        if (pq->tasks[i].priority > pq->tasks[highest_idx].priority) {
            highest_idx = i;
        }
    }

    int result_val = pq->tasks[highest_idx].val;

    for (int i = highest_idx; i < pq->count - 1; i++) {
        pq->tasks[i] = pq->tasks[i + 1];
    }
    pq->count--;

    return result_val;
}

```

```

#ifndef QUEUE_SINGLE_H
#define QUEUE_SINGLE_H

#define MAX_CAPACITY 100

```

```

typedef struct {
    int val;
    int priority;
} Task;

```

```

typedef struct {
    Task tasks[MAX_CAPACITY];
    int count;
} PriorityQueue;

```

```

void init_pq(PriorityQueue *pq);
int is_pq_empty(PriorityQueue *pq);
int is_pq_full(PriorityQueue *pq);
void pq_enqueue(PriorityQueue *pq, int val, int priority);
int pq_dequeue(PriorityQueue *pq);

```

```

#endif

```

(II)

```
#include <stdio.h>
#include "q2_queue_multi.h"

int main() {
    MultiPriorityQueue mpq;
    init_multi_pq(&mpq);

    printf("Enqueue 2 with Medium Priority (2)\n");
    mpq_enqueue(&mpq, 2, 2);
    printf("Enqueue 1 with High Priority (3)\n");
    mpq_enqueue(&mpq, 1, 3);
    printf("Enqueue 3 with Low Priority (1)\n");
    mpq_enqueue(&mpq, 3, 1);
    printf("Enqueue 4 with High Priority (3)\n");
    mpq_enqueue(&mpq, 4, 3);

    while (!mpq_is_empty(&mpq)) {
        printf("Processing Task: %d\n", mpq_dequeue(&mpq));
    }

    return 0;
}

#include <stdio.h>
#include "q2_queue_multi.h"

void init_queue(Queue *q) {
    q->front = -1;
    q->rear = -1;
}

int is_empty(Queue *q) {
    return q->front == -1;
}

int is_full(Queue *q) {
    return (q->rear + 1) % MAX_CAPACITY == q->front;
}

void enqueue(Queue *q, int val) {
    if (is_full(q)) return;
    if (is_empty(q)) {
        q->front = 0;
        q->rear = 0;
    } else {
        q->rear = (q->rear + 1) % MAX_CAPACITY;
    }
    q->data[q->rear] = val;
}

int dequeue(Queue *q) {
    if (is_empty(q)) return -1;
    int val = q->data[q->front];
    if (q->front == q->rear) {
        q->front = -1;
        q->rear = -1;
    } else {
        q->front = (q->front + 1) % MAX_CAPACITY;
    }
}
```

```

    return val;
}

void init_multi_pq(MultiPriorityQueue *mpq) {
    init_queue(&mpq->high);
    init_queue(&mpq->medium);
    init_queue(&mpq->low);
}

void mpq_enqueue(MultiPriorityQueue *mpq, int val, int priority) {
    if (priority == 3) {
        enqueue(&mpq->high, val);
    } else if (priority == 2) {
        enqueue(&mpq->medium, val);
    } else if (priority == 1) {
        enqueue(&mpq->low, val);
    }
}

int mpq_dequeue(MultiPriorityQueue *mpq) {
    if (!is_empty(&mpq->high)) {
        return dequeue(&mpq->high);
    } else if (!is_empty(&mpq->medium)) {
        return dequeue(&mpq->medium);
    } else if (!is_empty(&mpq->low)) {
        return dequeue(&mpq->low);
    }
    return -1;
}

int mpq_is_empty(MultiPriorityQueue *mpq) {
    return is_empty(&mpq->high) && is_empty(&mpq->medium) && is_empty(&mpq->low);
}

```

```

#ifndef QUEUE_MULTI_H
#define QUEUE_MULTI_H

#define MAX_CAPACITY 100

```

```

typedef struct {
    int data[MAX_CAPACITY];
    int front;
    int rear;
} Queue;

```

```

typedef struct {
    Queue high;
    Queue medium;
    Queue low;
} MultiPriorityQueue;

```

```

void init_queue(Queue *q);
int is_empty(Queue *q);
int is_full(Queue *q);
void enqueue(Queue *q, int val);
int dequeue(Queue *q);

```

```

void init_multi_pq(MultiPriorityQueue *mpq);
void mpq_enqueue(MultiPriorityQueue *mpq, int val, int priority);

```

```

ChamalInduwara@MacBook-Air : ~/Downloads
$ gcc q2_main_multi.c q2_queue_multi.c

```

```

ChamalInduwara@MacBook-Air : ~/Downloads
$ ./a.out
Enqueue 2 with Medium Priority (2)
Enqueue 1 with High Priority (3)
Enqueue 3 with Low Priority (1)
Enqueue 4 with High Priority (3)
Processing Task: 1
Processing Task: 4
Processing Task: 2
Processing Task: 3

```



```
int mpq_dequeue(MultiPriorityQueue *mpq);  
int mpq_is_empty(MultiPriorityQueue *mpq);  
  
#endif
```